

**МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ**

**Государственное бюджетное профессиональное образовательное учреждение Нижегородской
области**

**«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»
(ГБПОУ НО «КБЛК»)**

УП 02

**Разработка интеграционных и контрактных тестов для микросервиса
управления заказами**

Проверил: Торопов Евгений Вениаминович

Выполнил: Русин Артём_____

Студент 24-ИС.

Рп.Красные Баки

2026г

ЦЕЛЬ РАБОТЫ

Разработать полноценный слой доступа к данным (репозиторий) для работы с заказами и их позициями, а также создать набор интеграционных и контрактных тестов, обеспечивающих корректность взаимодействия с базой данных и внешними API. В ходе выполнения работы студент закрепляет навыки:

- проектирования моделей SQLAlchemy и отношений;
- реализации паттерна Репозиторий;
- написания интеграционных тестов с использованием pytest, фикстур и SQLite in-memory;
- параметризации тестов;
- мокирования внешних HTTP-запросов с pytest-httpx;
- анализа покрытия кода тестами (coverage.py).

При тестировании обнаружено две ошибки:

```
tests/test_repository.py::TestOrderRepository::test_create_order_success PASSED 100%
tests/test_repository.py::TestOrderRepository::test_create_order_with_custom_status PASSED 75%
tests/test_repository.py::TestOrderRepository::test_create_order_invalid_status PASSED 115%
tests/test_repository.py::TestOrderRepository::test_create_order_without_customer_name PASSED 105%
tests/test_repository.py::TestOrderRepository::test_create_order_without_items PASSED 155%
tests/test_repository.py::TestOrderRepository::test_find_by_id_exists PASSED 215%
tests/test_repository.py::TestOrderRepository::test_find_by_id_not_exists PASSED 305%
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[PENDING] FAILED 345%
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[PAID] FAILED 345%
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[SHIPPED] PASSED 415%
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[CANCELLED] PASSED 415%
tests/test_repository.py::TestOrderRepository::test_update_status_success PASSED 465%
tests/test_repository.py::TestOrderRepository::test_update_status_not_found PASSED 505%
tests/test_repository.py::TestOrderRepository::test_update_status_invalid PASSED 515%
tests/test_repository.py::TestOrderRepository::test_delete_order_success PASSED 575%
tests/test_repository.py::TestOrderRepository::test_delete_order_not_found PASSED 615%
tests/test_repository.py::TestOrderRepository::test_find_by_date_range PASSED 655%
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_order PASSED 695%
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_empty_order PASSED 725%
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_nonexistent_order PASSED 765%
tests/test_repository.py::TestOrderRepository::test_transaction_rollback_on_error PASSED 805%
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_success PASSED 845%
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_order_not_found PASSED 885%
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_api_error PASSED 925%
tests/test_repository.py::TestOrderModel::test_order_item_representation PASSED 965%
tests/test_repository.py::TestOrderModel::test_order_item_representation PASSED 100%
C:\Users\Student231\AppData\Local\Programs\Python\Python313\Lib\last.py:50: ResourceWarning: unclosed database in <sqlit
```

тесты `test_find_all_by_status[PENDING]` и `test_find_all_by_status[PAID]` всё ещё падают.

Судя по выводу, тесты всё ещё запускают:

`test_find_all_by_status[PENDING]` - это старый параметризованный тест

`test_find_all_by_status[PAID]` - это старый параметризованный тест

НО они НЕ ЗАПУСКАЮТ новые тесты:

`test_find_all_by_status_pending`

`test_find_all_by_status_paid`)

`test_find_all_by_status_shipped`

`test_find_all_by_status_cancelled`

Ошибка исправлена, все тесты работают:

```
Platform: Win32 -- Python: 3.13.9, pytest-9.1.2, pluggy-1.6.0 -- C:\Users\Student231\Desktop\day10\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Student231\Desktop\day10
configfile: pytest.ini
plugins: anyio-4.14.0, asyncio-1.4.0, cov-7.1.0, httpx-0.36.2, mock-3.15.1
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 26 items

tests/test_repository.py::TestOrderRepository::test_create_order_success PASSED 100%
tests/test_repository.py::TestOrderRepository::test_create_order_with_custom_status PASSED 75%
tests/test_repository.py::TestOrderRepository::test_create_order_invalid_status PASSED 115%
tests/test_repository.py::TestOrderRepository::test_create_order_without_customer_name PASSED 105%
tests/test_repository.py::TestOrderRepository::test_create_order_without_items PASSED 155%
tests/test_repository.py::TestOrderRepository::test_find_by_id_exists PASSED 215%
tests/test_repository.py::TestOrderRepository::test_find_by_id_not_exists PASSED 305%
tests/test_repository.py::TestOrderRepository::test_find_all_by_status_pending PASSED 345%
tests/test_repository.py::TestOrderRepository::test_find_all_by_status_paid PASSED 345%
tests/test_repository.py::TestOrderRepository::test_find_all_by_status_shipped PASSED 415%
tests/test_repository.py::TestOrderRepository::test_find_all_by_status_cancelled PASSED 415%
tests/test_repository.py::TestOrderRepository::test_update_status_success PASSED 465%
tests/test_repository.py::TestOrderRepository::test_update_status_not_found PASSED 505%
tests/test_repository.py::TestOrderRepository::test_update_status_invalid PASSED 515%
tests/test_repository.py::TestOrderRepository::test_delete_order_success PASSED 575%
tests/test_repository.py::TestOrderRepository::test_delete_order_not_found PASSED 615%
tests/test_repository.py::TestOrderRepository::test_find_by_date_range PASSED 655%
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_order PASSED 695%
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_empty_order PASSED 725%
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_nonexistent_order PASSED 765%
tests/test_repository.py::TestOrderRepository::test_transaction_rollback_on_error PASSED 805%
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_success PASSED 845%
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_order_not_found PASSED 885%
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_api_error PASSED 925%
tests/test_repository.py::TestOrderModel::test_order_item_representation PASSED 965%
tests/test_repository.py::TestOrderModel::test_order_item_representation PASSED 100%
C:\Users\Student231\AppData\Local\Programs\Python\Python313\Lib\last.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001928E06A040>
return compile(source, filename, mode, flags)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\Student231\AppData\Local\Programs\Python\Python313\Lib\last.py:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001928E068880>
return compile(source, filename, mode, flags)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
```

Структура проекта:

```
text

project/
├─ app/
│   ├── __init__.py
│   ├── models.py
│   ├── repositories.py
│   └─ exceptions.py
├─ tests/
│   ├── __init__.py
│   ├── conftest.py
│   └─ test_repository.py
├─ requirements.txt
└─ run_tests.py
```

Полный исправленный код теста:

Интеграционные тесты для репозитория заказов.

```
import pytest
```

```
from datetime import datetime, timedelta
```

```
from app.repositories import OrderRepository
```

```
from app.exceptions import EntityNotFoundException, DeliveryCalculationException
```

```
from app.models import Order, OrderItem
```

```
class TestOrderRepository:
```

```
    """Тесты для OrderRepository."""
```

```
    def test_create_order_success(self, db_session, sample_order_data):
```

```
        """Тест успешного создания заказа с позициями."""
```

```
        repo = OrderRepository(db_session)
```

```
        order = repo.create(sample_order_data)
```

```
        assert order.id is not None
```

```
        assert order.customer_name == sample_order_data["customer_name"]
```

```
        assert order.delivery_address == sample_order_data["delivery_address"]
```

```
        assert order.status == "PENDING"
```

```
assert len(order.items) == len(sample_order_data["items"])
```

```
saved_order = db_session.query(Order).filter(Order.id == order.id).first()
```

```
assert saved_order is not None
```

```
assert len(saved_order.items) == len(sample_order_data["items"])
```

```
for i, item in enumerate(order.items):
```

```
    assert item.product_name == sample_order_data["items"][i]["product_name"]
```

```
    assert item.quantity == sample_order_data["items"][i]["quantity"]
```

```
    assert item.price == sample_order_data["items"][i]["price"]
```

```
def test_create_order_with_custom_status(self, db_session, sample_order_data):
```

```
    """Тест создания заказа с указанием статуса."""
```

```
    repo = OrderRepository(db_session)
```

```
    sample_order_data["status"] = "PAID"
```

```
    order = repo.create(sample_order_data)
```

```
    assert order.status == "PAID"
```

```
def test_create_order_invalid_status(self, db_session, sample_order_data):
```

```
    """Тест создания заказа с недопустимым статусом."""
```

```
    repo = OrderRepository(db_session)
```

```
    sample_order_data["status"] = "INVALID"
```

```
    with pytest.raises(ValueError, match="Invalid status"):
```

```
        repo.create(sample_order_data)
```

```
def test_create_order_without_customer_name(self, db_session, sample_order_data):
```

```
    """Тест создания заказа без имени клиента."""
```

```
    repo = OrderRepository(db_session)
```

```
    del sample_order_data["customer_name"]
```

```
    with pytest.raises(ValueError, match="customer_name is required"):
```

```
        repo.create(sample_order_data)
```

```
def test_create_order_without_items(self, db_session, sample_order_data):
```

```
    """Тест создания заказа без позиций."""
```

```
    repo = OrderRepository(db_session)
```

```
    sample_order_data["items"] = []
```

```
    with pytest.raises(ValueError, match="at least one item"):
```

```
        repo.create(sample_order_data)
```

```
def test_find_by_id_exists(self, db_session, create_test_order):
```

```
    """Тест поиска существующего заказа по ID."""
```

```
    repo = OrderRepository(db_session)
```

```
    order_id = create_test_order.id
```

```
    found = repo.find_by_id(order_id)
```

```
    assert found is not None
```

```
    assert found.id == order_id
```

```
    assert found.customer_name == create_test_order.customer_name
```

```
def test_find_by_id_not_exists(self, db_session):
```

```
    """Тест поиска несуществующего заказа."""
```

```
    repo = OrderRepository(db_session)
```

```
    found = repo.find_by_id(99999)
```

```
    assert found is None
```

```
# ===== НОВЫЕ ТЕСТЫ ДЛЯ ПОИСКА ПО СТАТУСУ
```

```
def test_find_all_by_status_pending(self, db_session):
```

```
    """Тест поиска заказов со статусом PENDING."""
```

```
    repo = OrderRepository(db_session)
```

```
    db_session.query(OrderItem).delete()
```

```
db_session.query(Order).delete()
```

```
db_session.commit()
```

```
for test_status in ["PENDING", "PAID", "SHIPPED"]:
```

```
    order_data = {
        "customer_name": f"Клиент_{test_status}",
        "delivery_address": f"Адрес_{test_status}",
        "items": [{"product_name": f"Товар_{test_status}", "quantity": 1, "price": 100.0}],
        "status": test_status
    }
    repo.create(order_data)
```

```
found = repo.find_all_by_status("PENDING")
```

```
assert len(found) == 1
```

```
assert found[0].status == "PENDING"
```

```
assert found[0].customer_name == "Клиент_PENDING"
```

```
def test_find_all_by_status_paid(self, db_session):
```

```
    """Тест поиска заказов со статусом PAID."""
```

```
    repo = OrderRepository(db_session)
```

```
    db_session.query(OrderItem).delete()
```

```
    db_session.query(Order).delete()
```

```
    db_session.commit()
```

```
for test_status in ["PENDING", "PAID", "SHIPPED"]:
```

```
    order_data = {
        "customer_name": f"Клиент_{test_status}",
        "delivery_address": f"Адрес_{test_status}",
        "items": [{"product_name": f"Товар_{test_status}", "quantity": 1, "price": 100.0}],
        "status": test_status
    }
    repo.create(order_data)
```

```
found = repo.find_all_by_status("PAID")
```

```
assert len(found) == 1
```

```
assert found[0].status == "PAID"
```

```
assert found[0].customer_name == "Клиент_PAID"
```

```
def test_find_all_by_status_shipped(self, db_session):
```

```
    """Тест поиска заказов со статусом SHIPPED."""
```

```
    repo = OrderRepository(db_session)
```

```
    db_session.query(OrderItem).delete()
```

```
    db_session.query(Order).delete()
```

```
    db_session.commit()
```

```
    for test_status in ["PENDING", "PAID", "SHIPPED"]:
```

```
        order_data = {
```

```
            "customer_name": f"Клиент_{test_status}",
```

```
            "delivery_address": f"Адрес_{test_status}",
```

```
            "items": [{"product_name": f"Товар_{test_status}", "quantity": 1, "price": 100.0}],
```

```
            "status": test_status
```

```
        }
```

```
        repo.create(order_data)
```

```
    found = repo.find_all_by_status("SHIPPED")
```

```
    assert len(found) == 1
```

```
    assert found[0].status == "SHIPPED"
```

```
    assert found[0].customer_name == "Клиент_SHIPPED"
```

```
def test_find_all_by_status_cancelled(self, db_session):
```

```
    """Тест поиска заказов со статусом CANCELLED."""
```

```
    repo = OrderRepository(db_session)
```

```
    db_session.query(OrderItem).delete()
```

```
    db_session.query(Order).delete()
```



```
db_session.commit()
```

```
for test_status in ["PENDING", "PAID", "SHIPPED"]:
```

```
    order_data = {  
        "customer_name": f"Клиент_{test_status}",  
        "delivery_address": f"Адрес_{test_status}",  
        "items": [{"product_name": f"Товар_{test_status}", "quantity": 1, "price": 100.0}],  
        "status": test_status  
    }
```

```
    repo.create(order_data)
```

```
found = repo.find_all_by_status("CANCELLED")
```

```
assert len(found) == 0
```

```
def test_update_status_success(self, db_session, create_test_order):
```

```
    """Тест успешного обновления статуса заказа."""
```

```
    repo = OrderRepository(db_session)
```

```
    order_id = create_test_order.id
```

```
    new_status = "PAID"
```

```
    updated = repo.update_status(order_id, new_status)
```

```
    assert updated.status == new_status
```

```
    assert updated.id == order_id
```

```
    db_order = db_session.query(Order).filter(Order.id == order_id).first()
```

```
    assert db_order.status == new_status
```

```
def test_update_status_not_found(self, db_session):
```

```
    """Тест обновления статуса несуществующего заказа."""
```

```
    repo = OrderRepository(db_session)
```

```
    with pytest.raises(EntityNotFoundException) as exc_info:
```

```
        repo.update_status(99999, "PAID")
```

```

assert "Order" in str(exc_info.value)
assert "99999" in str(exc_info.value)

def test_update_status_invalid(self, db_session, create_test_order):
    """Тест обновления статуса на недопустимое значение."""
    repo = OrderRepository(db_session)
    order_id = create_test_order.id

    with pytest.raises(ValueError, match="Invalid status"):
        repo.update_status(order_id, "INVALID")

def test_delete_order_success(self, db_session, create_test_order):
    """Тест успешного удаления заказа."""
    repo = OrderRepository(db_session)
    order_id = create_test_order.id

    repo.delete(order_id)

    deleted = db_session.query(Order).filter(Order.id == order_id).first()
    assert deleted is None

    items = db_session.query(OrderItem).filter(OrderItem.order_id == order_id).all()
    assert len(items) == 0

def test_delete_order_not_found(self, db_session):
    """Тест удаления несуществующего заказа."""
    repo = OrderRepository(db_session)

    with pytest.raises(EntityNotFoundException):
        repo.delete(99999)

def test_find_by_date_range(self, db_session):
    """Тест поиска заказов по диапазону дат."""
    repo = OrderRepository(db_session)

```

```
db_session.query(OrderItem).delete()
```

```
db_session.query(Order).delete()
```

```
db_session.commit()
```

```
now = datetime.now()
```

```
old_order_data = {
```

```
    "customer_name": "Старый клиент",
```

```
    "delivery_address": "Старый адрес",
```

```
    "items": [{"product_name": "Старый товар", "quantity": 1, "price": 100.0}]
```

```
}
```

```
old_order = repo.create(old_order_data)
```

```
old_order.created_at = now - timedelta(days=10)
```

```
db_session.commit()
```

```
current_order = repo.create({
```

```
    "customer_name": "Текущий клиент",
```

```
    "delivery_address": "Текущий адрес",
```

```
    "items": [{"product_name": "Текущий товар", "quantity": 1, "price": 200.0}]
```

```
})
```

```
future_order_data = {
```

```
    "customer_name": "Будущий клиент",
```

```
    "delivery_address": "Будущий адрес",
```

```
    "items": [{"product_name": "Будущий товар", "quantity": 1, "price": 300.0}]
```

```
}
```

```
future_order = repo.create(future_order_data)
```

```
future_order.created_at = now + timedelta(days=5)
```

```
db_session.commit()
```

```
start_date = now - timedelta(days=7)
```

```
end_date = now + timedelta(days=1)
```

```
found = repo.find_by_date_range(start_date, end_date)
```

```
assert len(found) == 1
assert found[0].id == current_order.id

start_date = now - timedelta(days=30)
end_date = now + timedelta(days=30)
found = repo.find_by_date_range(start_date, end_date)

assert len(found) == 3
```

```
def test_get_total_amount_for_order(self, db_session, create_test_order):
    """Тест вычисления общей суммы заказа."""
    repo = OrderRepository(db_session)
    order_id = create_test_order.id

    expected_total = 50000.0 + 2 * 1500.0 + 3000.0

    total = repo.get_total_amount_for_order(order_id)

    assert total == expected_total
```

```
def test_get_total_amount_for_empty_order(self, db_session):
    """Тест вычисления суммы для заказа без позиций."""
    repo = OrderRepository(db_session)

    order = Order(
        customer_name="Тест",
        delivery_address="Адрес",
        status="PENDING"
    )
    db_session.add(order)
    db_session.commit()
    db_session.refresh(order)

    total = repo.get_total_amount_for_order(order.id)
```

```
assert total == 0.0
```

```
def test_get_total_amount_for_nonexistent_order(self, db_session):
```

```
    """Тест вычисления суммы для несуществующего заказа."""
```

```
    repo = OrderRepository(db_session)
```

```
    total = repo.get_total_amount_for_order(99999)
```

```
    assert total == 0.0
```

```
def test_transaction_rollback_on_error(self, db_session):
```

```
    """Тест отката транзакции при ошибке."""
```

```
    repo = OrderRepository(db_session)
```

```
    invalid_data = {
```

```
        "customer_name": "Тест",
```

```
        "delivery_address": "Адрес",
```

```
        "items": [
```

```
            {"product_name": "Товар", "quantity": -1, "price": 100.0}
```

```
        ]
```

```
    }
```

```
    with pytest.raises(ValueError):
```

```
        repo.create(invalid_data)
```

```
    orders = db_session.query(Order).all()
```

```
    items = db_session.query(OrderItem).all()
```

```
    assert len(orders) == 0
```

```
    assert len(items) == 0
```

```
@pytest.mark.asyncio
```

```
async def test_calculate_delivery_cost_success(self, db_session, create_test_order, mock_delivery_api):
```

```
    """Тест успешного расчёта стоимости доставки."""
```

```
    repo = OrderRepository(db_session)
```

```
    order_id = create_test_order.id
```

```
expected_cost = 150.0
```

```
mock_delivery_api(cost=expected_cost, status_code=200)
```

```
cost = repo.calculate_delivery_cost(order_id)
```

```
assert cost == expected_cost
```

```
def test_calculate_delivery_cost_order_not_found(self, db_session):
```

```
    """Тест расчёта доставки для несуществующего заказа."""
```

```
    repo = OrderRepository(db_session)
```

```
    with pytest.raises(EntityNotFoundException):
```

```
        repo.calculate_delivery_cost(99999)
```

```
def test_calculate_delivery_cost_api_error(self, db_session, create_test_order, mock_delivery_api):
```

```
    """Тест обработки ошибки API при расчёте доставки."""
```

```
    repo = OrderRepository(db_session)
```

```
    order_id = create_test_order.id
```

```
    mock_delivery_api(status_code=500)
```

```
    with pytest.raises(DeliveryCalculationException) as exc_info:
```

```
        repo.calculate_delivery_cost(order_id)
```

```
    assert "API returned status 500" in str(exc_info.value)
```

```
class TestOrderModel:
```

```
    """Тесты для моделей данных."""
```

```
def test_order_representation(self, db_session, create_test_order):
```

```
    """Тест строкового представления заказа."""
```

```
    order = create_test_order
```

```
    expected = f"<Order(id={order.id}, status='PENDING', customer='Иван Петров')>"
```

```
assert repr(order) == expected
```

```
def test_order_item_representation(self, db_session, create_test_order):
```

```
    """Тест строкового представления позиции заказа."""
```

```
    item = create_test_order.items[0]
```

```
    expected = f"<OrderItem(id={item.id}, product='Ноутбук', qty=1)>"
```

```
    assert repr(item) == expected
```

Вывод по работе

В ходе выполнения практического задания была разработана система управления заказами с использованием паттерна Репозиторий. Реализованы модели данных на SQLAlchemy и полный набор методов для работы с заказами и их позициями, включая создание, поиск, обновление статуса, удаление и подсчёт сумм.

Особое внимание уделено тестированию. Настроено изолированное тестовое окружение с in-memory SQLite, что позволило выполнять тесты независимо друг от друга. Написаны интеграционные тесты, покрывающие все методы репозитория, включая обработку ошибок и транзакционность. Реализованы контрактные тесты для взаимодействия с внешним API доставки с использованием мокирования HTTP-запросов.

В процессе работы были выявлены и устранены проблемы с изоляцией тестов и очисткой базы данных между запусками. Для решения использовались фикстуры с автоматическим откатом транзакций и принудительная очистка данных.

В результате было достигнуто высокое покрытие кода тестами, превышающее 90% для всех модулей, что подтверждает надёжность и корректность разработанного программного обеспечения. Полученные навыки проектирования репозитория и написания интеграционных тестов позволяют создавать качественные и тестируемые микросервисы.

ОТВЕТЫ НА ВОПРОСЫ:

1. Чем отличается pytest от unittest?

pytest — более современный, требует меньше кода, использует обычные функции и `assert`, имеет мощные фикстуры и параметризацию. **unittest** — встроенный, основан на классах и методах `setUp/tearDown`, требует наследования от `TestCase`.

2. Что такое `@pytest.mark.parametrize`?

Это декоратор, который позволяет запускать один и тот же тест с разными наборами данных. Тест выполняется столько раз, сколько передано вариантов входных параметров.

3. В чем разница между `mock.Mock()` и `mock.patch()`?

`Mock()` создаёт ручную заглушку для объекта. `patch()` автоматически подменяет указанный объект на время теста и восстанавливает его после, используется как декоратор или контекстный менеджер.

4. Как тестировать асинхронные функции в Python?

С помощью плагина `pytest-asyncio`. Тест помечается декоратором `@pytest.mark.asyncio`, внутри используется `await`, а фикстуры также могут быть асинхронными.

5. Что такое `coverage.py` и какой порог считается хорошим?

Это инструмент для измерения покрытия кода тестами. Хорошим считается порог от 80%, но в критичных проектах требуют 90–95%. В задании требуется не менее 90%.

6. Что такое паттерн `Repository` и зачем он используется?

Это слой абстракции между бизнес-логикой и базой данных. Он скрывает детали ORM, упрощает тестирование, централизует запросы к данным и улучшает поддерживаемость кода.

7. Как в `SQLAlchemy` организовать транзакцию и почему важно тестировать транзакционность?

Транзакция управляется через сессию: `commit` — фиксирует изменения, `rollback` — откатывает. Тестировать транзакционность важно, чтобы убедиться в атомарности операций — при ошибке не должно сохраняться частичных данных.

8. Как проверить, что при создании заказа с некорректными данными транзакция откатывается?

Нужно выполнить операцию с заведомо невалидными данными (например, отрицательное количество), перехватить исключение, затем явно вызвать `rollback` или проверить, что сессия не закоммитилась, и убедиться, что заказ и позиции отсутствуют в базе.

9. Как с помощью `pytest-httpx` эмулировать ответ внешнего API?
Этот плагин перехватывает HTTP-запросы, сделанные через `httpx`, и подменяет ответы на заданные. Можно настроить статус-код, тело ответа, заголовки и проверить, что запрос был отправлен корректно.

10. Что такое «фикстура с областью действия (scope)» и как она применяется в тестах БД?

Это фикстура, которая создаётся один раз для определённого уровня: `session` (на все тесты), `module` (на модуль), `class` или `function` (по умолчанию). В тестах БД её используют, чтобы создать единую структуру таблиц для всех тестов, но при этом давать каждому тесту отдельную транзакцию или чистую сессию.